

PyTorch Autograd

Modern Computer Vision | Spring 2026

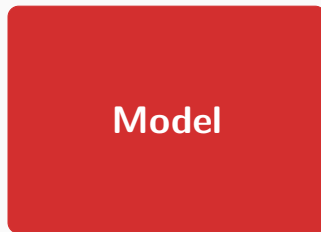
Oren Chikli | Tutorial 3

This presentation largely relies on DL4CV@Weizmann 23'

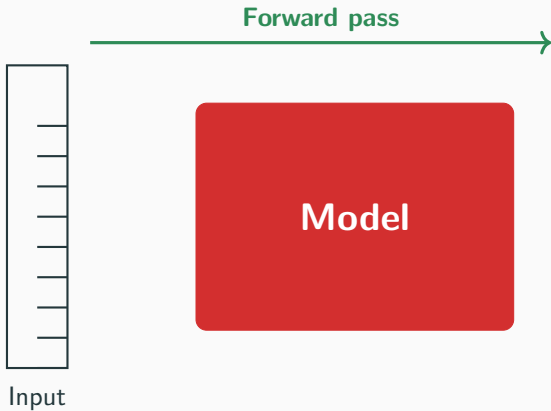
Neural network — high-level pipeline



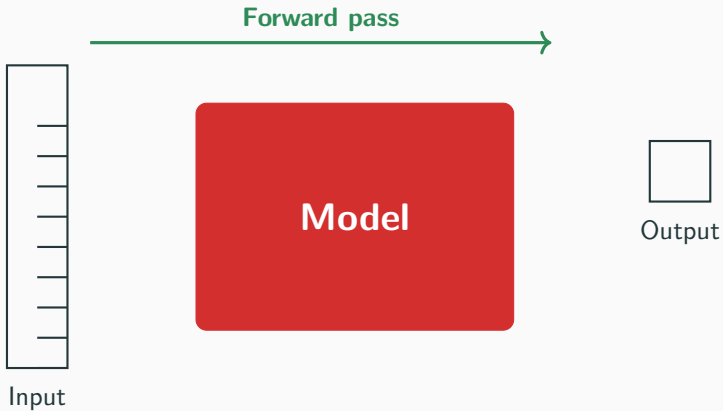
Input



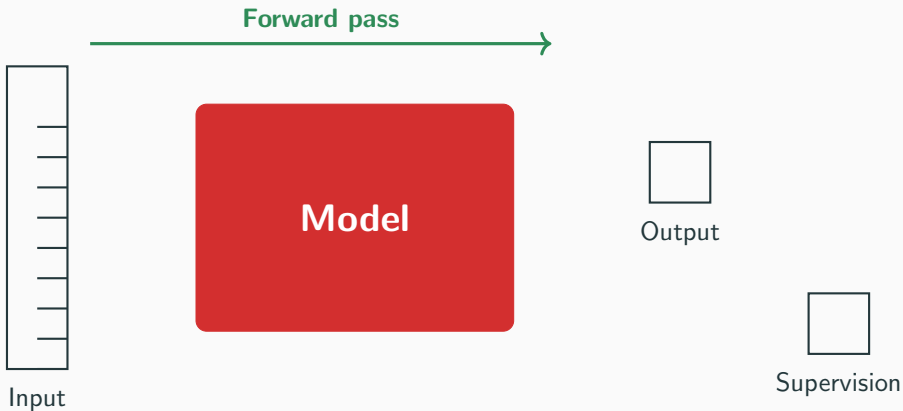
Neural network — high-level pipeline



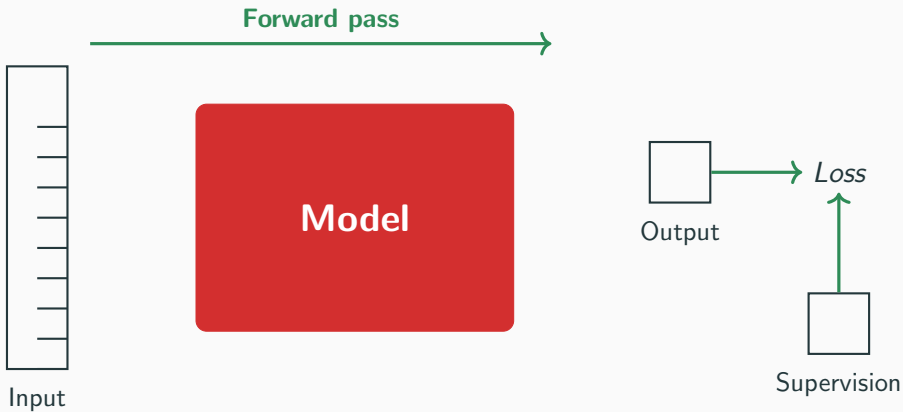
Neural network — high-level pipeline



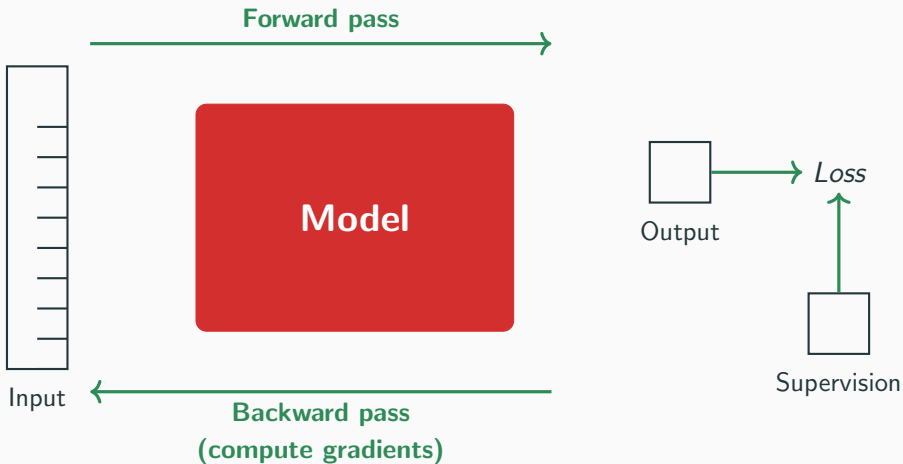
Neural network — high-level pipeline



Neural network — high-level pipeline



Neural network — high-level pipeline



Building a Neural Network — a network is a Python class

A network has learnable state (its parameters), so it's naturally a *stateful object* — a Python class. In PyTorch, subclass `nn.Module`.

```
class MyNetwork(nn.Module):
    def __init__(self):          # runs once at construction
        super().__init__()
        # declare your layers
    def forward(self, x):        # runs every model(x) call
        # describe how data flows
        return ...
```

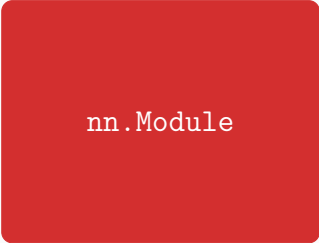
Building a Neural Network — a network is a Python class

A network has learnable state (its parameters), so it's naturally a *stateful object* — a Python class. In PyTorch, subclass `nn.Module`.

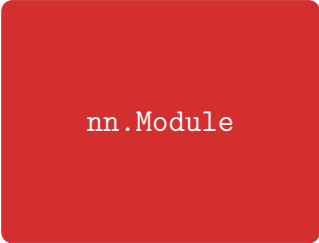
```
class MyNetwork(nn.Module):
    def __init__(self):          # runs once at construction
        super().__init__()
        # declare your layers
    def forward(self, x):        # runs every model(x) call
        # describe how data flows
        return ...
```

By subclassing `nn.Module` you get for free:

- Layers declared in `__init__` are auto-registered — `model.parameters()` sees them.
- Recursive `.to(device)`, `.train()`, `.eval()` across every sub-module.



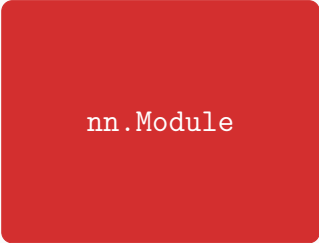
`nn.Module`



`nn.Module`

`__init__()`

`__init__()` declares the layers (parameters).



`nn.Module`

`__init__()`

`forward()`

`__init__()` declares the layers (parameters).
`forward()` composes them into a computation.

Building a Neural Network

```
INPUT_SIZE = 224
C = 10 # num classes

class MLP(nn.Module):

    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(256, 512)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(512, C)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x
```

Building a Neural Network

```
INPUT_SIZE = 224
C = 10 # num classes

class MLP(nn.Module):

    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(256, 512)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(512, C)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x
```

Initialize

```
B = 10 # batch size

input = torch.randn(B, 1, INPUT_SIZE, INPUT_SIZE)
model = MLP()
```

input shape: $(B, 1, H, W)$

output shape: (B, C)

Building a Neural Network

```
INPUT_SIZE = 224
C = 10 # num classes

class MLP(nn.Module):

    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(256, 512)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(512, C)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x
```

Initialize

```
B = 10 # batch size

input = torch.randn(B, 1, INPUT_SIZE, INPUT_SIZE)
model = MLP()
```

input shape: $(B, 1, H, W)$

output shape: (B, C)

Run with CPU

```
output = model(input)
```

7.96 milliseconds

Run with GPU

```
gpu_model = model.to('cuda')
gpu_input = input.to('cuda')

gpu_output = gpu_model(gpu_input)
```

0.58 milliseconds

Building a Neural Network

```
INPUT_SIZE = 224
C = 10 # num classes

class MLP(nn.Module):

    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(256, 512)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(512, C)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x
```

Using Sequential

```
seq_model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(INPUT_SIZE*INPUT_SIZE, 256),
    nn.ReLU(),
    nn.Linear(256, 512),
    nn.ReLU(),
    nn.Linear(512, 10),
)

output = seq_model(input)
```

Same model, two ways.

`nn.Sequential` folds the `__init__` + simple-chain forward into a single declaration — works whenever the forward pass is just a sequence of calls.

Building a Neural Network

```
class ResNetLikeCNN(nn.Module):
    def __init__(self, num_classes):
        super(ResNetLikeCNN, self).__init__()

        self.features = nn.Sequential(
            ResBlock(3, 16, kernel_size=3, stride=1),
            ResBlock(16, 32, kernel_size=3, stride=2),
            ResBlock(32, 64, kernel_size=3, stride=2),
        )

        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(64, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

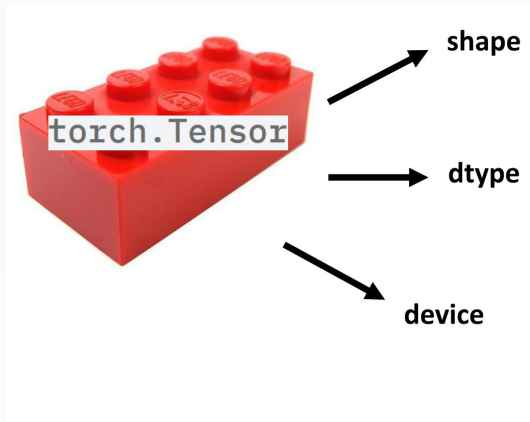
Reminder example

- **Data:** $x, y \in \mathbb{R}$
- **Hypothesis:** $\hat{y} = x \cdot w$
- **Loss:** $L = (\hat{y} - y)^2$
- **SGD update:** $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

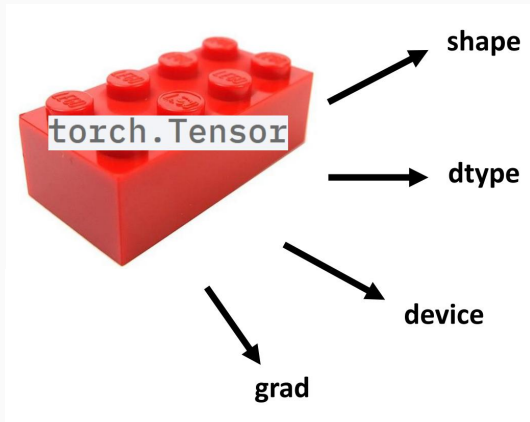
Reminder example

- **Data:** $x, y \in \mathbb{R}$
- **Hypothesis:** $\hat{y} = x \cdot w$
- **Loss:** $L = (\hat{y} - y)^2$
- **SGD update:** $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

Tensor attributes — autograd

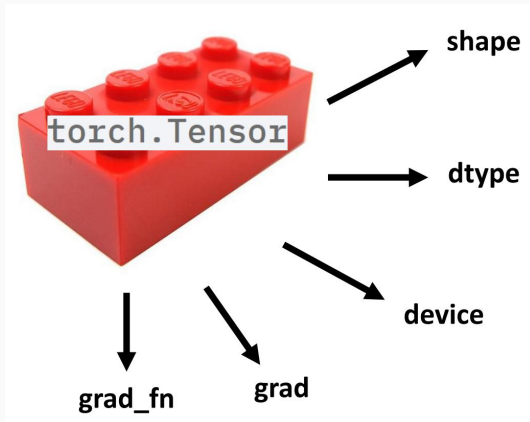


Tensor attributes — autograd



`.grad` – gradient for this tensor.

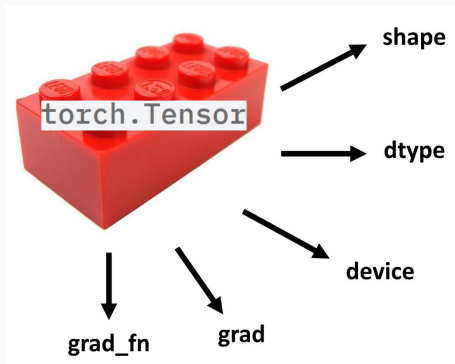
Tensor attributes — autograd



`.grad` – gradient for this tensor.

`.grad_fn` – reference for a function, used to obtain this tensor.

Tensor attributes — autograd



```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

```
print(w.grad)  
print(x.grad)
```

```
tensor([-4.])
```

```
None
```

Computational graph

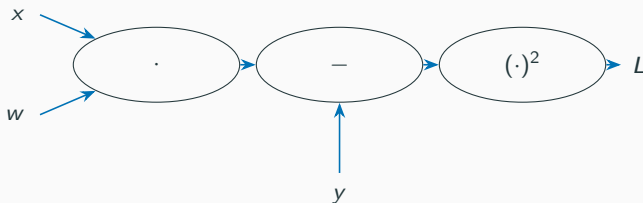
- A **directed graph** whose nodes correspond to operations or variables.

Computational graph

- A **directed graph** whose nodes correspond to operations or variables.
- Can be used to differentiate via the *chain rule*.

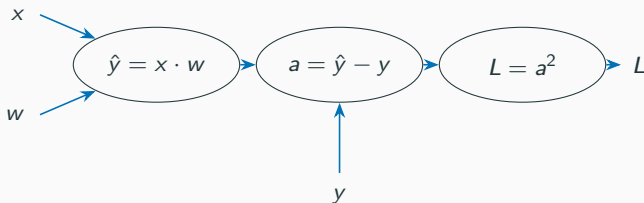
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



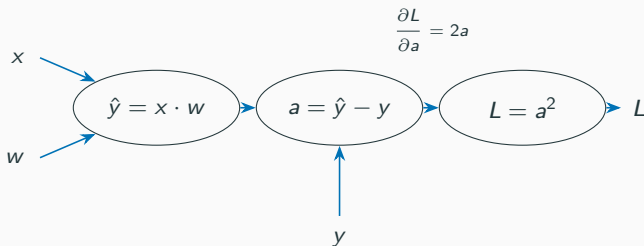
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



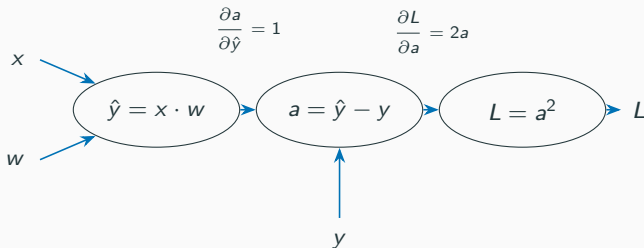
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



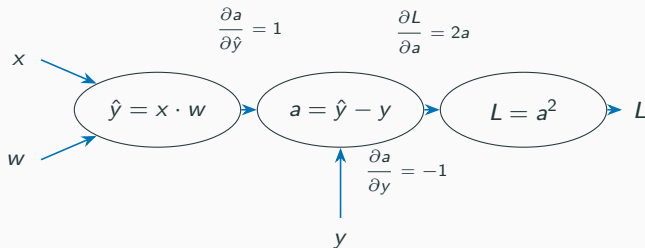
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



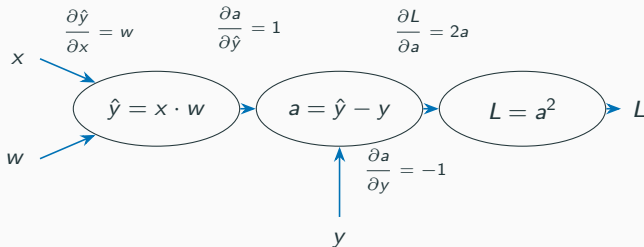
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



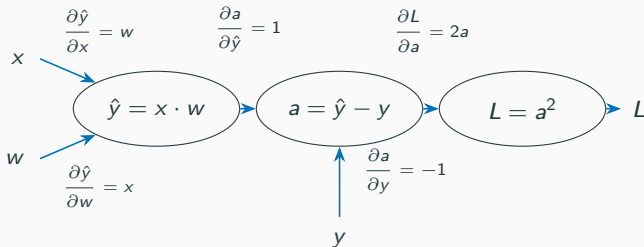
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



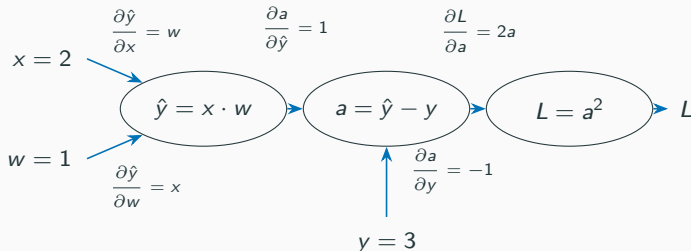
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



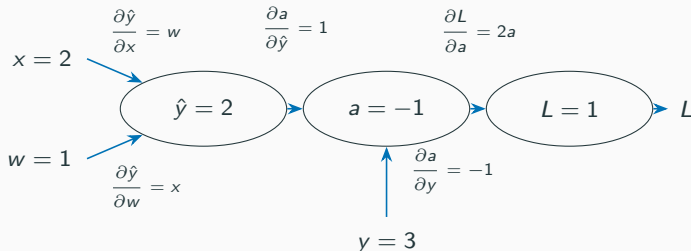
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



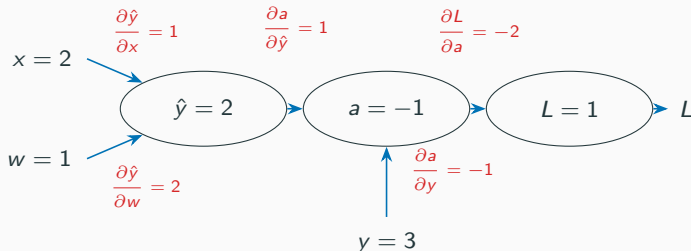
Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

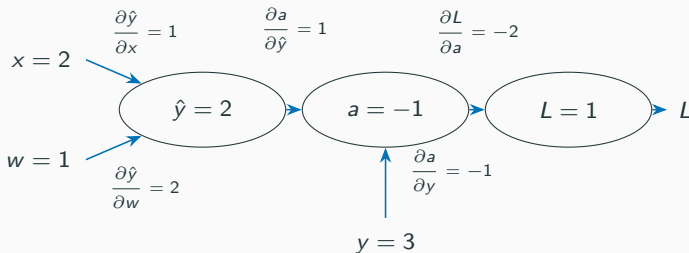


Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

Chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

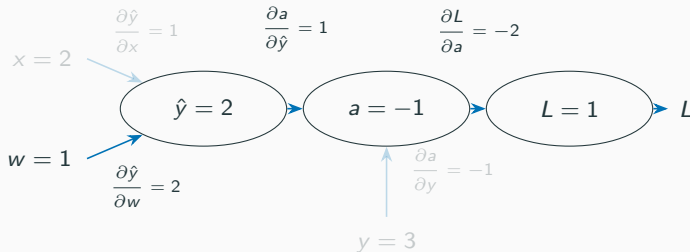


Computational Graph

- Data: $x, y \in \mathbb{R}$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

Chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$



Static vs Dynamic computational graph

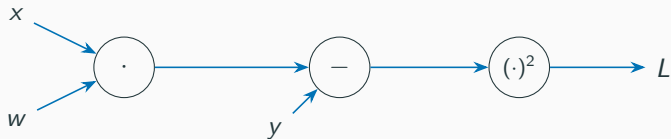
- **Static graph** — the graph's structure is *fixed* ahead of time; data is then *injected* into the pre-built graph.

Static vs Dynamic computational graph

- **Static graph** — the graph's structure is *fixed* ahead of time; data is then *injected* into the pre-built graph.
- **Dynamic graph** — the graph's structure is *defined on the fly*, as Python code runs.

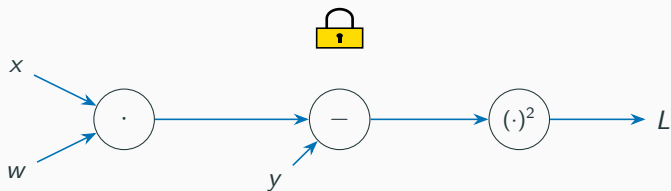
Static graph — define-and-run

First define the graph, **then** inject data.



Static graph — define-and-run

First define the graph, **then** inject data.



Graph structure is fixed before any data flows through it.

Dynamic graph — define-by-run

Variables and their relations are defined; the graph is built *implicitly* as the code runs.

x

w

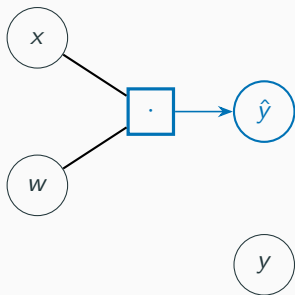
y

Code so far

```
x, w, y
```

Dynamic graph — define-by-run

Variables and their relations are defined; the graph is built *implicitly* as the code runs.

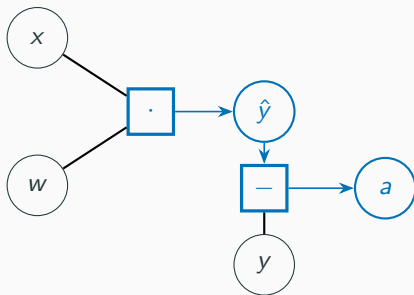


Code so far

```
x, w, y  
y_hat = x * w
```

Dynamic graph — define-by-run

Variables and their relations are defined; the graph is built *implicitly* as the code runs.

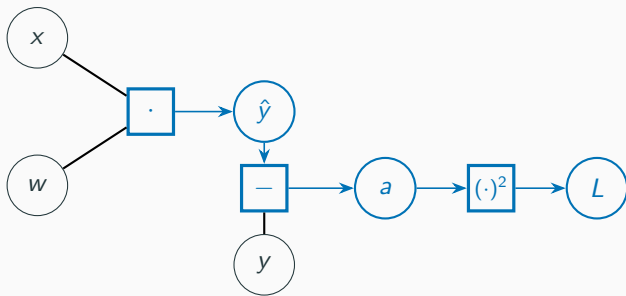


Code so far

```
x, w, y  
y_hat = x * w  
a = y_hat - y
```

Dynamic graph — define-by-run

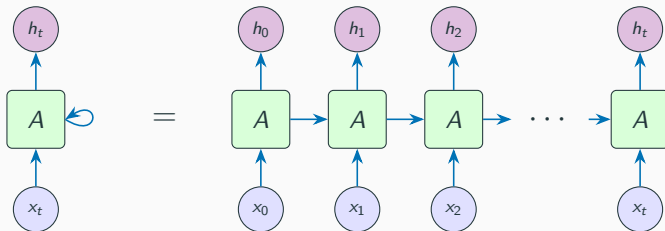
Variables and their relations are defined; the graph is built *implicitly* as the code runs.



Code so far

```
x, w, y  
y_hat = x * w  
a = y_hat - y  
L = a ** 2
```

Why dynamic 1/2 — variable-length sequences



```
def forward(self, words):                                # len(words) varies per input
    h = torch.zeros(self.hidden_dim)
    for word in words:
        h = tanh(Wx @ word + Wh @ h)
    return self.out(h)
```

The for-loop length *is* the graph depth.

Why dynamic 2/2 — adaptive-depth networks

```
def forward(self, x):  
    h = self.stem(x)  
    for layer in self.layers:  
        h = layer(h)  
        p = self.head(h).softmax(-1)  
        if p.max() > 0.95:                # data-dependent branch  
            return p  
    return self.head(h)
```

Easy images exit at layer 4; hard ones run all 20.

The graph follows Python control flow, which follows the *data*.

Static graphs bake every possible path in upfront (`tf.cond`, `tf.while_loop`).

Static vs Dynamic — at a glance

Static	Dynamic
--------	---------

Static vs Dynamic — at a glance

Static	Dynamic
define-and-run	

Static vs Dynamic — at a glance

Static	Dynamic
define-and-run	define-by-run

Static vs Dynamic — at a glance

Static	Dynamic
define-and-run	define-by-run
slightly faster	

Static vs Dynamic — at a glance

Static	Dynamic
define-and-run	define-by-run
slightly faster	flexible

Static vs Dynamic — at a glance

Static	Dynamic
define-and-run	define-by-run
slightly faster	flexible
 TensorFlow < 2.0	

Static vs Dynamic — at a glance

Static	Dynamic
define-and-run	define-by-run
slightly faster	flexible
 TensorFlow < 2.0	 PyTorch

Static vs Dynamic — at a glance

Static	Dynamic
define-and-run	define-by-run
slightly faster	flexible
 TensorFlow < 2.0	 PyTorch
	 TensorFlow ≥ 2.0 (eager)

- **Dynamic.**

Computational graph in PyTorch

- **Dynamic.**
- Created *on the fly* during the forward pass.

Computational graph in PyTorch

- **Dynamic.**
- Created *on the fly* during the forward pass.
 - Unless `torch.no_grad()` was used.

Computational graph in PyTorch

- **Dynamic.**
- Created *on the fly* during the forward pass.
 - Unless `torch.no_grad()` was used.
- **Deleted** during the backward pass (memory is freed once gradients flow back).

Computational graph in PyTorch

- **Dynamic.**
- Created *on the fly* during the forward pass.
 - Unless `torch.no_grad()` was used.
- **Deleted** during the backward pass (memory is freed once gradients flow back).
- Re-built from scratch on the next forward pass.

Computational Graph in PyTorch

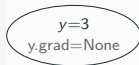
```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])  
  
y_hat = x * w  
L = (y_hat - y) ** 2  
  
L.backward()
```

Computational Graph in PyTorch

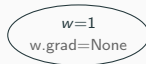
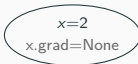
```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```



► leaves

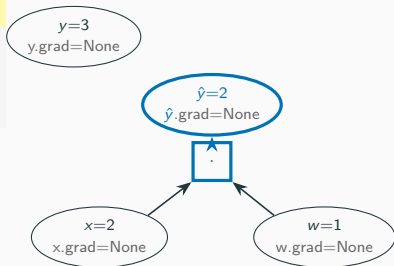


Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2  
  
L.backward()
```

► **y_hat**

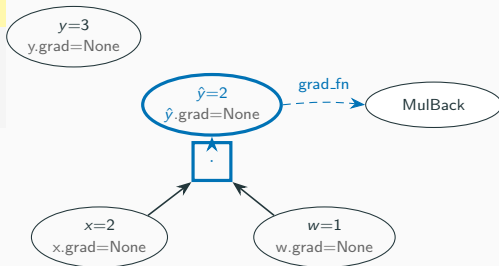


Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2  
  
L.backward()
```

► **y_hat**

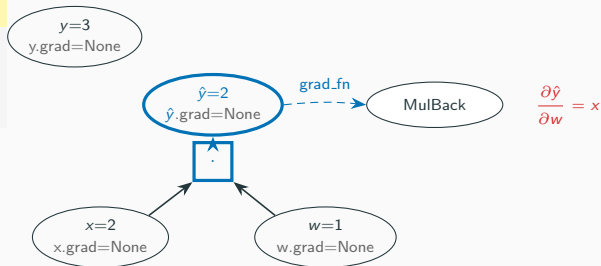


Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2  
  
L.backward()
```

► **y_hat**



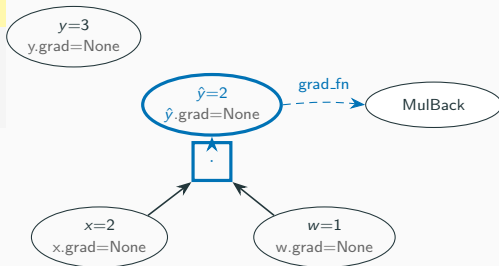
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► **y_hat**



$$\frac{\partial \hat{y}}{\partial w} = x$$



Computational Graph in PyTorch

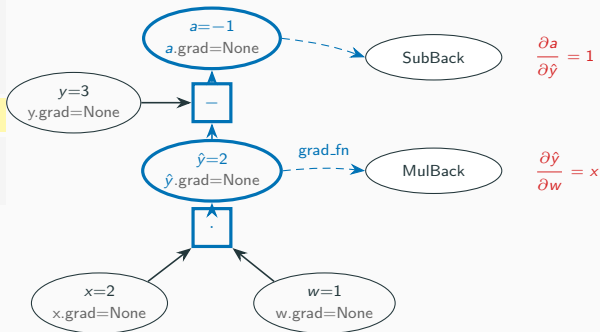
```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w
```

```
L = (y_hat - y) ** 2
```

```
L.backward()
```

► a



Computational Graph in PyTorch

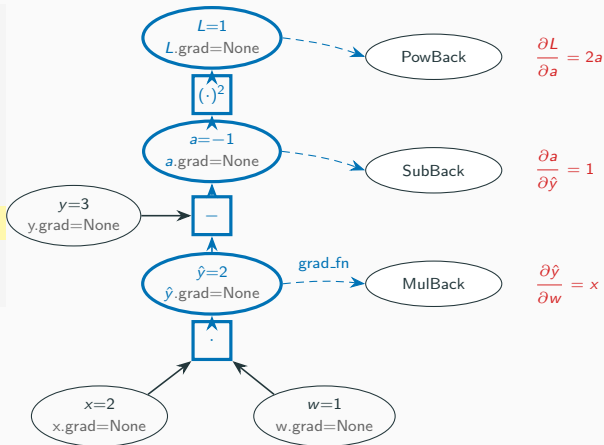
```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w
```

```
L = (y_hat - y) ** 2
```

```
L.backward()
```

► L



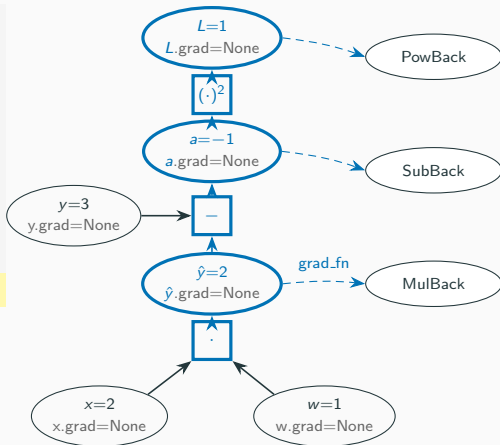
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



a
x
ctx

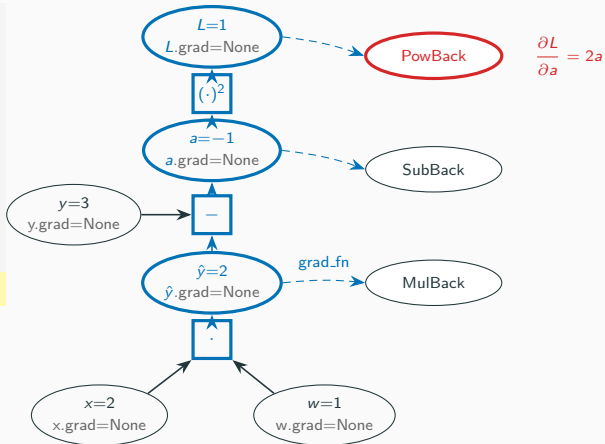
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



a
x
ctx

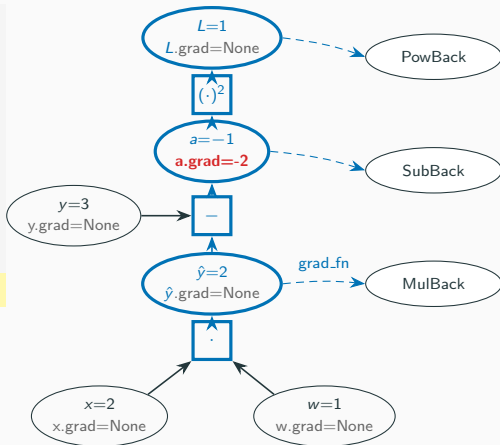
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



$$\frac{\partial L}{\partial a} = 2a = -2$$

x
ctx

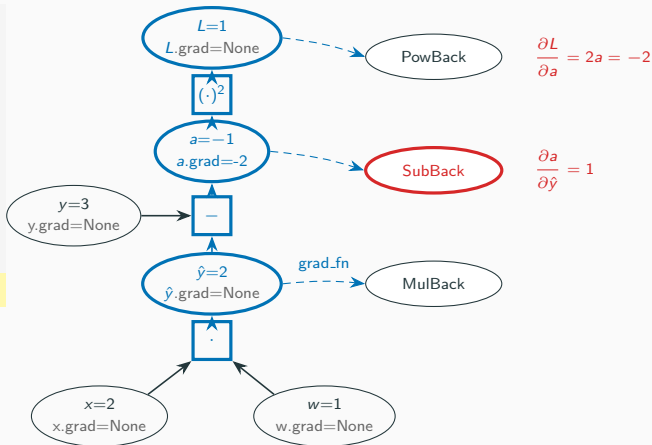
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                  requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



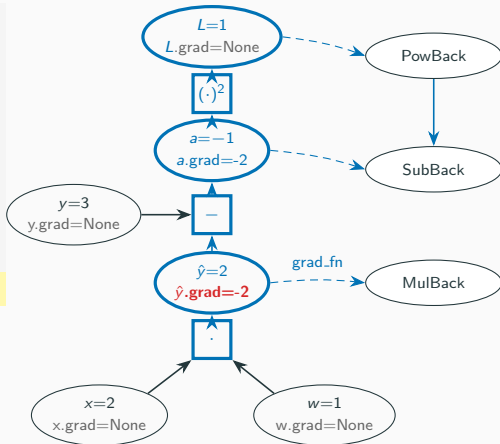
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



$$\frac{\partial L}{\partial a} = 2a = -2$$

$$\begin{aligned}\frac{\partial L}{\partial \hat{y}} &= \frac{\partial L}{\partial a} \frac{\partial a}{\partial \hat{y}} \\ &= 2a \cdot 1 \\ &= -2\end{aligned}$$

x
ctx

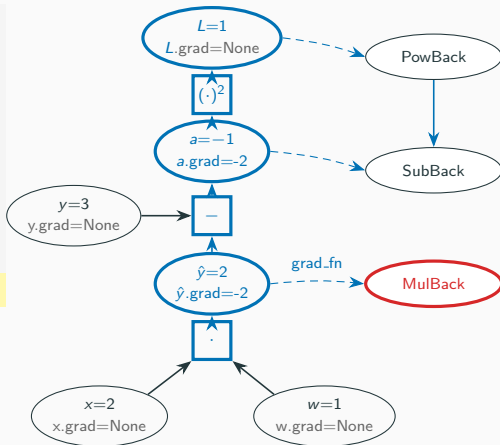
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



x
ctx

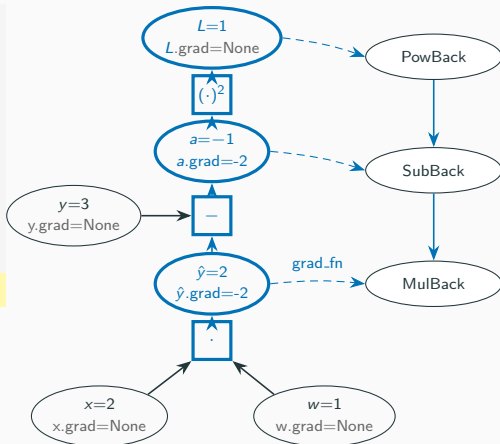
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



x
ctx

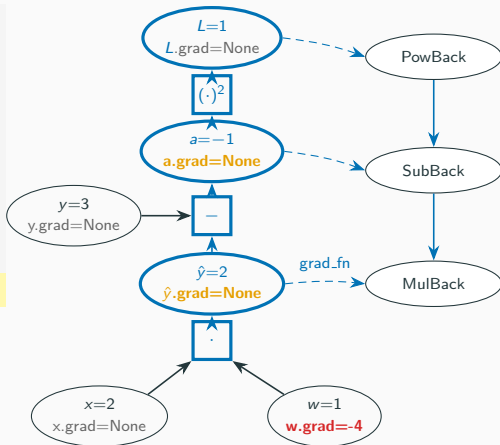
Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

► backward



$$\frac{\partial L}{\partial a} = 2a = -2$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial \hat{y}} = 2a \cdot 1$$

$$= -2$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w} = 2a x$$

$$= -4$$

PyTorch frees intermediate `.grad` after `backward()` — use `.retain_grad()` to keep them.

Computational Graph in PyTorch

```
x = torch.tensor([2.])  
w = torch.tensor([1.],  
                 requires_grad=True)  
y = torch.tensor([3.])
```

```
y_hat = x * w  
L = (y_hat - y) ** 2
```

```
L.backward()
```

```
print(w.grad)  
print(x.grad)
```

```
tensor([-4.])
```

```
None
```



Computational Graph in PyTorch

```
x = torch.tensor([2.])
w = torch.tensor([1.],
                 requires_grad=True)
y = torch.tensor([3.])

y_hat = x * w
L = (y_hat - y) ** 2

L.backward()

L = (x * w - y) ** 2
L.backward()

print(w.grad)
```



Gradients *accumulate* across `backward()` calls.

Gradient accumulation — a real use case

Problem. You want `batch_size = 128`, but only `batch_size = 32` fits in GPU memory.

Trick. Run 4 forward + backward passes *without* `zero_grad()`; gradients accumulate to what one big batch would produce. `step()` once.

```
ACCUM_STEPS = 4    # effective batch = 32 * 4 = 128

optimizer.zero_grad()
for i, (X, y) in enumerate(loader):
    loss = criterion(model(X), y) / ACCUM_STEPS    # rescale
    loss.backward()                                # accumulates
    if (i + 1) % ACCUM_STEPS == 0:
        optimizer.step()
        optimizer.zero_grad()
```

Disabling gradients

When do you not want autograd?

Inference. Validation. Statistics on inputs / weights. Anything where the result is consumed by a human, not by `backward()`.

`torch.no_grad()`

```
with torch.no_grad():  
    pred = model(x)
```

No graph constructed. Tensors still *exist*; they just can't be differentiated through.

`tensor.detach()`

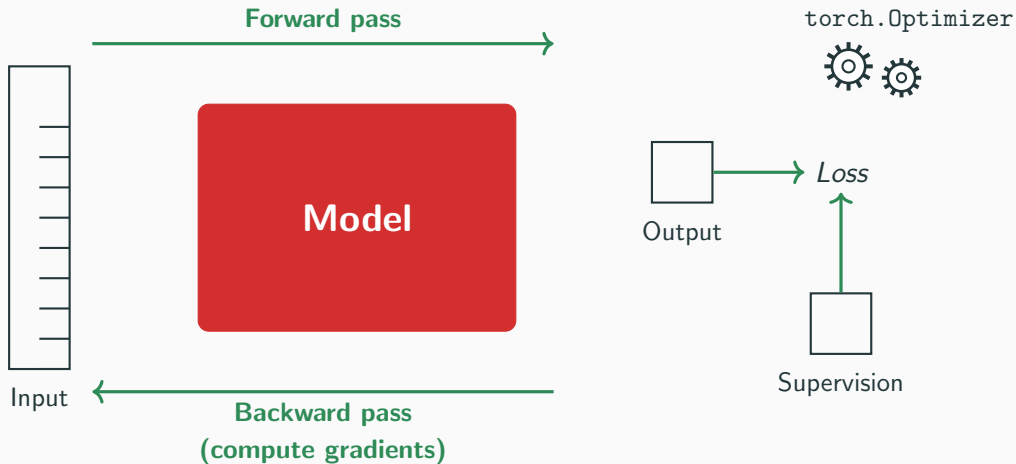
```
z = y.detach()
```

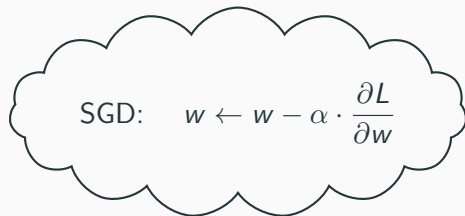
Returns a view of `y` that's not connected to the graph. Useful when you want to keep the value but cut the gradient path.

Back to NumPy

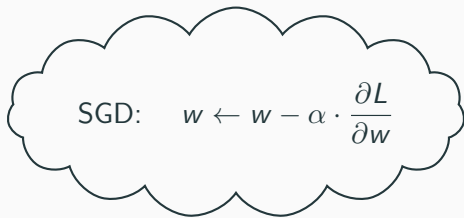
`tensor.detach().cpu().numpy()` — three steps: detach from graph, move to CPU, hand off as NumPy array.

Backpropagation — with the optimizer in the picture



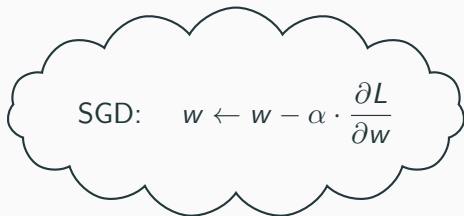


SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$


$$\text{SGD: } w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$$

Create an SGD optimizer

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

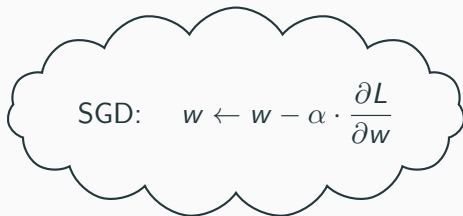

$$\text{SGD: } w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$$

Create an SGD optimizer

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Reset gradients

```
optimizer.zero_grad()
```



SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

Create an SGD optimizer

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Reset gradients

```
optimizer.zero_grad()
```

Take one optimization step

```
optimizer.step()
```

Putting it all together — Fashion MNIST



Putting it all together — the recipe

1. Hyperparameters
2. Handling data
3. Model, loss, optimizer
4. Training and inference
5. Overall training process

1. Hyperparameters

```
# configurations
LR = 1e-3           # learning rate
BATCH_SIZE = 64     # batch size
EPOCHS = 30         # num epochs
INPUT_SIZE = 28      # input size (28x28 grayscale)
NUM_WORKERS = 2      # dataloader workers

device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

2. Handling Data — Dataset

Dataset *“read one sample”*

```
training_data = datasets.FashionMNIST(  
    root="data",  
    train=True,  
    download=True,  
    transform=ToTensor(),  
)  
  
val_data = datasets.FashionMNIST(  
    root="data",  
    train=False,  
    download=True,  
    transform=ToTensor(),  
)
```

2. Handling Data — DataLoader

DataLoader *“batch + shuffle them”*

```
train_loader = DataLoader(  
    training_data,  
    batch_size=BATCH_SIZE,  
    shuffle=True,  
    num_workers=NUM_WORKERS,  
)  
  
val_loader = DataLoader(  
    val_data,  
    batch_size=BATCH_SIZE,  
    shuffle=False,  
    num_workers=NUM_WORKERS,  
)
```

3. Model, Loss, Optimizer

```
# define model
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(INPUT_SIZE*INPUT_SIZE, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 10),
)

# move to correct device
model.to(device)
```

```
# define loss
criterion = nn.CrossEntropyLoss()

# define optimizer
optimizer = torch.optim.SGD(
    model.parameters(), lr=LR)
```

4. Training and Inference

Training Loop

```
def train_loop(loader, model,
               criterion, optimizer):
    model.train()
    for X, y in loader:
        # move data to device
        X = X.to(device)
        y = y.to(device)
        # forward pass
        pred = model(X)
        loss = criterion(pred, y)
        # backward pass + update
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

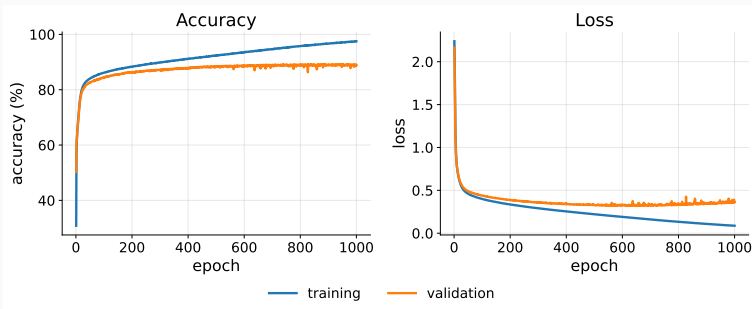
Inference Loop

```
@torch.no_grad()
def inference_loop(loader, model,
                  criterion):
    model.eval()
    for X, y in loader:
        X = X.to(device)
        y = y.to(device)
        # forward only - no graph
        pred = model(X)
        loss = criterion(pred, y)
```

model.eval() → Dropout/BN inference mode | *@torch.no_grad() → no graph, saves memory*

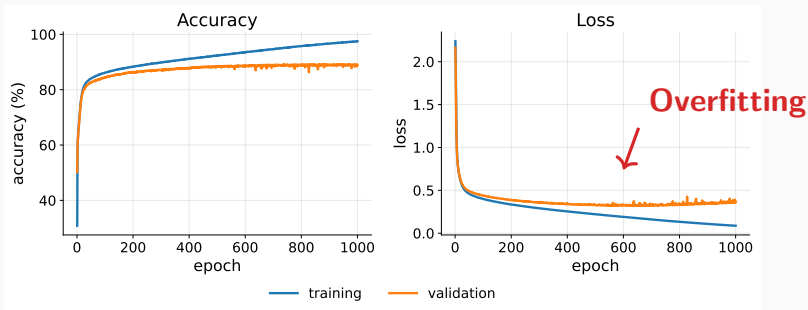
5. Overall Training Process

```
for t in range(EPOCHS):  
    train_loop(train_loader, model, criterion, optimizer)  
    inference_loop(val_loader, model, criterion)
```

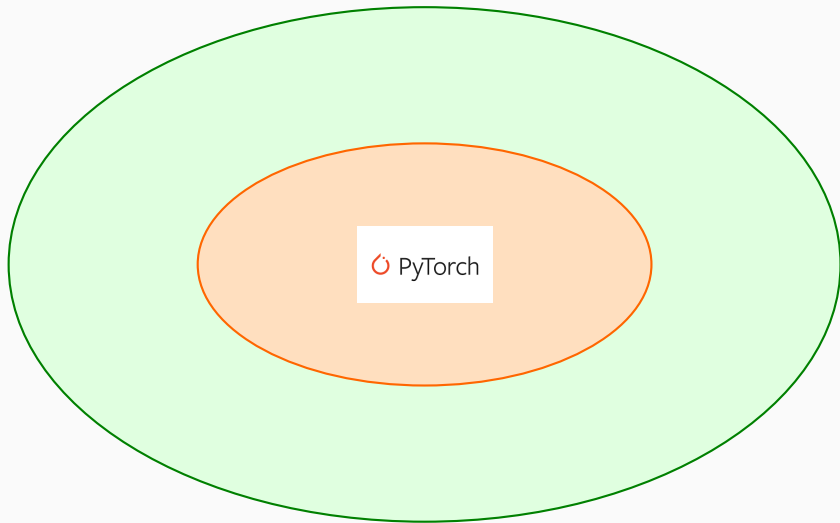


5. Overall Training Process

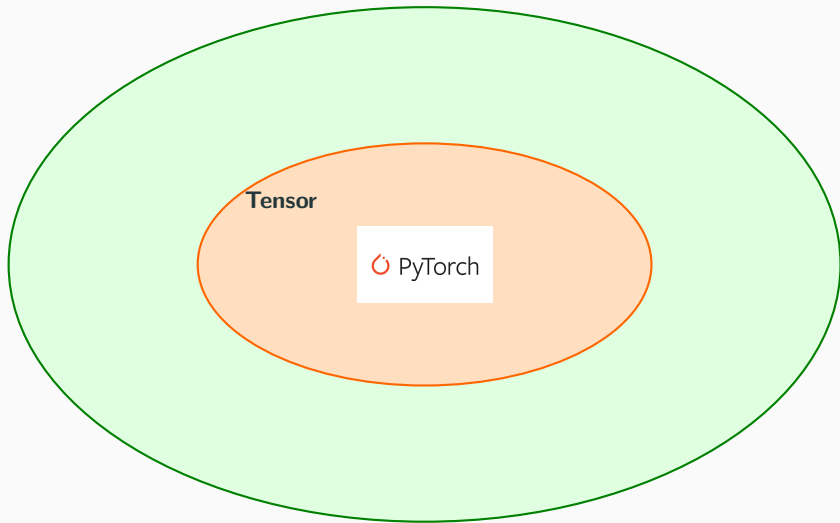
```
for t in range(EPOCHS):  
    train_loop(train_loader, model, criterion, optimizer)  
    inference_loop(val_loader, model, criterion)
```



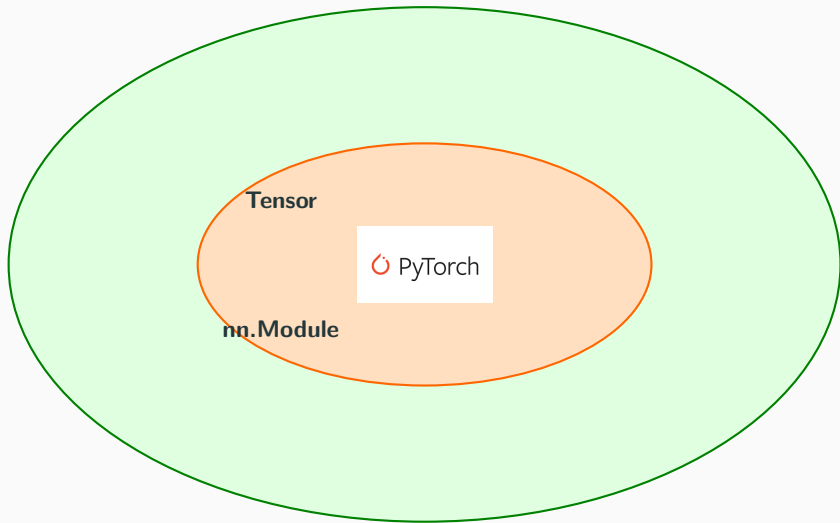
PyTorch ecosystem



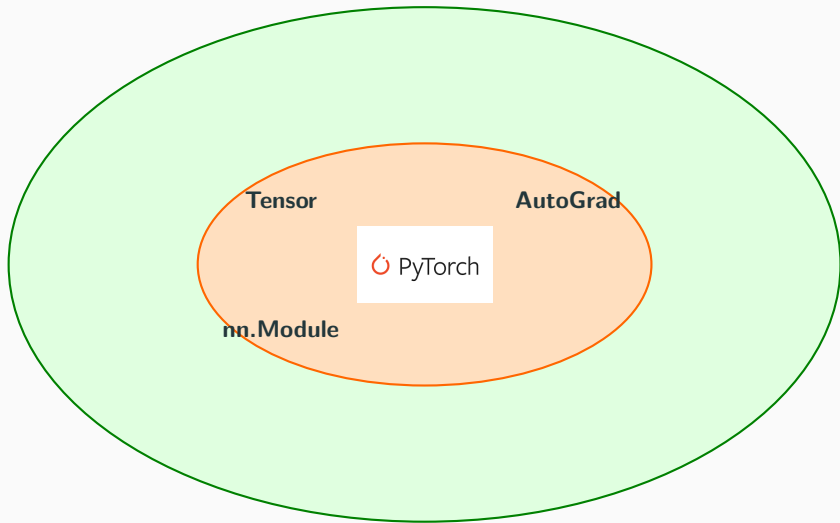
PyTorch ecosystem



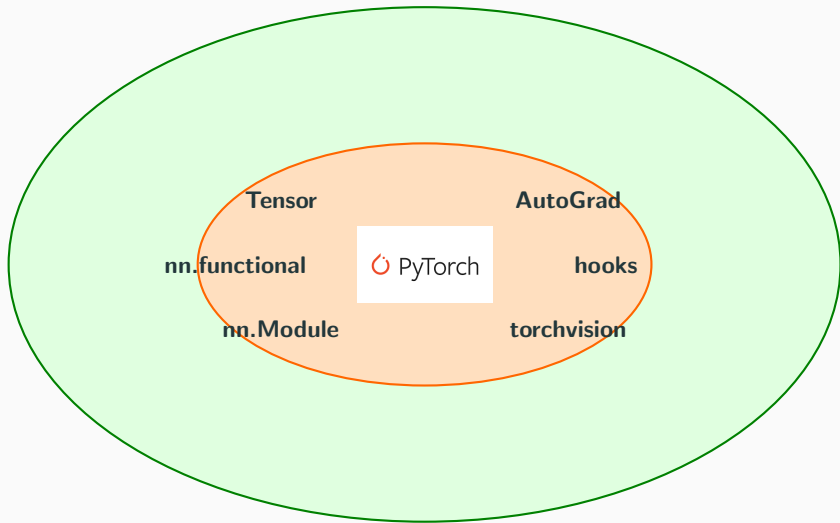
PyTorch ecosystem



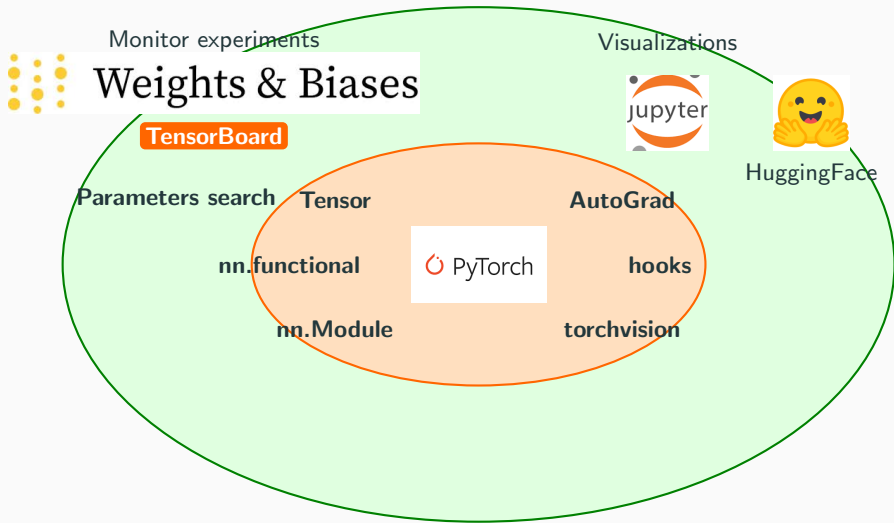
PyTorch ecosystem



PyTorch ecosystem



PyTorch ecosystem



PyTorch ecosystem

